

Universidad Nacional de Costa Rica

Escuela de Informática

Estructura de Datos

Sistema de Clustering Jerárquico

Dendrograma y Clasificación de Películas

Segundo Proyecto

Integrantes:

Andrey Jesús Solano Rojas

Wilson José Umaña Ríos

Gabriel Sánchez

Profesor:

Prof. Steven Brenes Chavarría

Noviembre, 2025

Índice

1. Introducción	4
1.1. Objetivos del Proyecto	4
2. Marco Teórico	4
2.1. ¿Qué es un Dendrograma?	4
2.1.1. Componentes de un Dendrograma	4
2.1.2. Aplicaciones	4
2.2. Clustering Jerárquico Aglomerativo	5
2.2.1. Average Linkage	5
3. Técnicas de Normalización	5
3.1. Normalización Min-Max	5
3.2. Estandarización Z-Score	6
3.3. Transformación Logarítmica	6
4. Métricas de Distancia	6
4.1. Distancia Euclidiana	6
4.2. Distancia Manhattan	7
4.3. Distancia Coseno	7
4.4. Distancia de Hamming	7
5. Arquitectura del Software	7
5.1. Estructura de Carpetas	7
5.2. Flujo de Datos	8
6. Patrones de Diseño Implementados	9
6.1. Patrón MVC (Model-View-Controller)	9
6.2. Patrón Singleton	10
6.3. Patrón Iterator	10
6.4. Patrón Strategy	11
6.4.1. Estrategias de Distancia	11
6.4.2. Estrategias de Normalización	11
6.5. Patrón Factory Method	11
7. Estructuras de Datos Implementadas	12
7.1. CustomList (Lista Enlazada)	12
7.2. CustomVector (Arreglo Dinámico)	12
7.3. CustomMatrix (Matriz Bidimensional)	13
7.4. CustomHashMap (Tabla Hash)	13

8. Análisis de Complejidad Algorítmica	13
8.1. Parseo de CSV	13
8.2. Vectorización de Películas	15
8.3. Normalización del Dataset	15
8.4. Construcción de Matriz de Distancias	15
8.5. Algoritmo de Clustering Jerárquico	16
8.6. Average Linkage - Cálculo de Distancia	16
8.7. Exportación a JSON	17
9. Construcción del Dendrograma	17
9.1. Matriz de Distancias	17
9.2. Proceso de Fusión de Clusters	18
9.3. Average Linkage	18
9.4. Reconstrucción de la Matriz	19
10. Sistema de Configuración	19
10.1. ConfigurationManager (Singleton)	19
10.2. Aplicación de Pesos	19
10.3. Configuración de Variables Categóricas	20
11. Operaciones Vectoriales	20
11.1. Producto Punto	20
11.2. Norma Euclidiana	20
11.3. Resta Vectorial	20
12. Interfaz de Usuario	21
12.1. Diseño de la Interfaz	21
12.2. Flujo de Interacción	21
13. Formato de Exportación JSON	22
13.1. Estructura del JSON	22
13.2. Características del Formato	22
14. Pruebas y Validación	23
14.1. Casos de Prueba	23
14.1.1. Prueba 1: Dataset Pequeño	23
14.1.2. Prueba 2: Comparación de Métricas	23
14.1.3. Prueba 3: Efecto de Ponderación	23
14.2. Validación de Resultados	24
15. Optimizaciones Implementadas	24
15.1. Vectorización One-Hot Eficiente	24
15.2. Matriz Simétrica	24
15.3. Crecimiento Exponencial de Vectores	24

16.Limitaciones y Trabajo Futuro	24
16.1. Limitaciones Actuales	24
17.Conclusiones	25
17.1. Aprendizajes Clave	25
17.2. Aplicabilidad	25

1. Introducción

El análisis de grandes volúmenes de datos es fundamental en la era de la información. Una de las técnicas más poderosas para descubrir patrones ocultos y relaciones entre elementos es el **clustering jerárquico**, método que permite agrupar datos basándose en similitudes sin necesidad de conocer previamente el número de grupos.

Este proyecto implementa un sistema completo de clustering jerárquico aplicado al dominio cinematográfico, permitiendo descubrir patrones de similitud entre películas basándose en múltiples características como presupuesto, popularidad, géneros, reparto y más. La visualización de estos agrupamientos se realiza mediante un **dendrograma**, representación gráfica que muestra la estructura jerárquica de las relaciones.

1.1. Objetivos del Proyecto

- Implementar un sistema de clustering jerárquico desde cero
- Aplicar múltiples métricas de distancia y técnicas de normalización
- Desarrollar estructuras de datos personalizadas eficientes
- Implementar patrones de diseño de software robustos
- Generar dendrogramas exportables en formato JSON

2. Marco Teórico

2.1. ¿Qué es un Dendrograma?

Un **dendrograma** es una representación gráfica en forma de árbol que ilustra la jerarquía de agrupamiento entre elementos de un conjunto de datos. Cada nivel del árbol representa una distancia o nivel de similitud al cual se fusionaron dos grupos.

2.1.1. Componentes de un Dendrograma

- **Hojas:** Representan elementos individuales del dataset
- **Nodos internos:** Representan la fusión de dos clusters
- **Altura:** Indica la distancia a la cual se fusionaron clusters
- **Ramas:** Conectan clusters fusionados

2.1.2. Aplicaciones

Los dendrogramas son ampliamente utilizados en:

- **Biología:** Análisis filogenético y taxonomía

- **Marketing:** Segmentación de clientes
- **Ciencias Sociales:** Clasificación de comportamientos
- **Minería de datos:** Descubrimiento de patrones
- **Entretenimiento:** Sistemas de recomendación

2.2. Clustering Jerárquico Aglomerativo

El método aglomerativo (bottom-up) sigue este proceso:

1. Inicializar cada elemento como un cluster individual
2. Calcular distancias entre todos los pares de clusters
3. Fusionar los dos clusters más cercanos
4. Actualizar matriz de distancias
5. Repetir hasta tener un único cluster

2.2.1. Average Linkage

El criterio **Average Linkage** calcula la distancia entre dos clusters como el promedio de las distancias entre todos los pares de elementos:

$$d(C_A, C_B) = \frac{|C_A| \cdot d(C_A, C_k) + |C_B| \cdot d(C_B, C_k)}{|C_A| + |C_B|} \quad (1)$$

donde $|C_A|$ y $|C_B|$ son los tamaños de los clusters A y B, y d es la distancia entre clusters.

3. Técnicas de Normalización

La normalización es crucial para asegurar que variables con diferentes escalas contribuyan equitativamente al análisis de clustering.

3.1. Normalización Min-Max

Transforma los valores a un rango específico, típicamente $[0, 1]$:

$$f(x) = \frac{x - \min(X)}{\max(X) - \min(X)} \quad (2)$$

Cuándo usar: Variables con la misma escala (edades, fechas, salarios).

Ventajas:

- Preserva la forma de la distribución original
- Fácil de interpretar
- Mantiene relaciones de distancia proporcionales

3.2. Estandarización Z-Score

Transforma los datos para tener media 0 y desviación estándar 1:

$$f(x) = \frac{x - \mu}{\sigma} \quad (3)$$

donde μ es la media y σ la desviación estándar.

Cuándo usar: Variables con diferentes unidades o escalas (cm vs m, kg vs g).

Ventajas:

- Maneja bien outliers moderados
- Permite comparación entre variables de diferentes escalas
- Útil cuando se asume distribución normal

3.3. Transformación Logarítmica

Aplica logaritmo natural para reducir el impacto de valores extremos:

$$f(x) = \log(x + 1) \quad (4)$$

Cuándo usar: Datos muy sesgados o con valores extremos (distribuciones no normales).

Ventajas:

- Reduce el efecto de outliers significativos
- Convierte distribuciones asimétricas en más simétricas
- Útil para datos con crecimiento exponencial

4. Métricas de Distancia

Las métricas de distancia cuantifican la similitud o diferencia entre vectores de características.

4.1. Distancia Euclidiana

Mide la distancia en línea recta entre dos puntos en un espacio n-dimensional:

$$d(x, y) = \sqrt{\sum_{i=0}^n (x_i - y_i)^2} \quad (5)$$

Aplicaciones: Recomendada como métrica general, especialmente cuando todas las dimensiones tienen importancia similar.

4.2. Distancia Manhattan

Suma las diferencias absolutas en cada dimensión:

$$d(x, y) = \sum_{i=0}^n |x_i - y_i| \quad (6)$$

Aplicaciones: Útil en espacios donde el movimiento está restringido a ejes (como calles en una ciudad), común en redes neuronales.

4.3. Distancia Coseno

Mide el ángulo entre dos vectores, no su magnitud:

$$d(x, y) = 1 - \frac{x \cdot y}{||x|| \cdot ||y||} \quad (7)$$

Aplicaciones: Ideal para comparar documentos de texto, sistemas de recomendación, cuando la dirección es más importante que la magnitud.

4.4. Distancia de Hamming

Cuenta el número de posiciones donde vectores discretos difieren:

$$d(x, y) = \sum_{i=0}^n \mathbb{K}_{x_i \neq y_i} \quad (8)$$

Aplicaciones: Detección de errores, genética, datos binarios o categóricos.

5. Arquitectura del Software

5.1. Estructura de Carpetas

El proyecto sigue una organización modular basada en responsabilidades:

Project02-Dendrogram/

Models/	# Modelos de datos
Movie.cs	# Entidad película
Cluster.cs	# Nodo del dendrograma
ClusterJSON.cs	# Modelo para exportación
DataStructures/	# Estructuras de datos propias
CustomList.cs	
CustomVector.cs	
CustomMatrix.cs	
CustomHashMap.cs	
Interfaces/	


```
    IIterable.cs
    IIterator.cs

Services/                                # Lógica de negocio
    CSVParser.cs                        # Parseo de archivos
    VectorizationService.cs            # Vectorización
    CategoryIndexer.cs                 # Indexación one-hot
    ConfigurationManager.cs            # Configuración singleton
    DendogramBuilder.cs                 # Construcción del árbol
    JSONExporter.cs                     # Exportación

Strategies/                              # Patrones Strategy
    Distance/                           # Métricas de distancia
        IDistanceStrategy.cs
        EuclideanDistance.cs
        ManhattanDistance.cs
        CosineDistance.cs
        HammingDistance.cs
        DistanceFactory.cs
    Normalization/                       # Estrategias de normalización
        INormalizationStrategy.cs
        MinMaxNormalization.cs
        ZScoreNormalization.cs
        LogNormalization.cs
        NormalizationFactory.cs

Presentation/                            # Capa de presentación (MVC)
    ClusteringJerarquico.xaml           # Vista
    ClusteringJerarquico.xaml.cs        # Vista code-behind
    ClusteringModel.cs                  # Modelo
    ClusteringController.cs             # Controlador

Utils/                                   # Utilidades
    VectorOperations.cs                  # Operaciones vectoriales
```

5.2. Flujo de Datos

El sistema procesa los datos siguiendo un pipeline de transformación:

1. **Carga de Datos:** CSV $\rightarrow$ Lista de películas
2. **Vectorización:** Películas $\rightarrow$ Vectores numéricos
3. **Normalización:** Vectores $\rightarrow$ Vectores normalizados
4. **Ponderación:** Aplicación de pesos configurados

5. **Clustering:** Construcción de dendrograma
6. **Exportación:** Dendrograma → JSON

CSV File

CSVParser

VectorizationService

ConfigurationManager
(Normalización +
Ponderación)

DendogramBuilder

JSONExporter

Figura 1: Pipeline de procesamiento de datos

6. Patrones de Diseño Implementados

6.1. Patrón MVC (Model-View-Controller)

Separa la lógica de presentación, datos y control:

- **Model** (ClusteringModel.cs): Contiene el estado de la aplicación

- **View** (ClusteringJerarquico.xaml): Interfaz gráfica WPF
- **Controller** (ClusteringController.cs): Coordina operaciones

Beneficios:

- Separación clara de responsabilidades
- Facilita pruebas unitarias
- Permite cambios en la UI sin afectar lógica de negocio

6.2. Patrón Singleton

ConfigurationManager implementa Singleton para mantener configuración única:

```
1 internal class ConfigurationManager
2 {
3     private static readonly Lazy<ConfigurationManager> _instance =
4         new Lazy<ConfigurationManager>(() => new ConfigurationManager());
5
6     public static ConfigurationManager Instance => _instance.Value;
7
8     private ConfigurationManager()
9     {
10         InitializeDefaultConfiguration();
11     }
12 }
```

Razón: Garantiza una única instancia de configuración en toda la aplicación.

6.3. Patrón Iterator

Permite recorrer colecciones sin exponer su estructura interna:

```
1 public interface IIterator<T>
2 {
3     bool HasNext();
4     T Next();
5     void Reset();
6 }
7
8 public interface IIterable<T>
9 {
10     IIterator<T> CreateIterator();
11 }
```

Implementaciones:

- ListIterator: Recorre listas enlazadas
- VectorIterator: Recorre vectores dinámicos
- HashMapIterator: Recorre tablas hash

6.4. Patrón Strategy

Encapsula algoritmos intercambiables en tiempo de ejecución:

6.4.1. Estrategias de Distancia

```
1 interface IDistanceStrategy
2 {
3     double Calculate(CustomVector<double> v1, CustomVector<double> v2);
4 }
5
6 class EuclideanDistance : IDistanceStrategy { ... }
7 class ManhattanDistance : IDistanceStrategy { ... }
8 class CosineDistance : IDistanceStrategy { ... }
9 class HammingDistance : IDistanceStrategy { ... }
```

6.4.2. Estrategias de Normalización

```
1 interface INormalizationStrategy
2 {
3     void CalculateStats(CustomList<Movie> movies, int featureIndex);
4     double Normalize(double value);
5 }
6
7 class MinMaxNormalization : INormalizationStrategy { ... }
8 class ZScoreNormalization : INormalizationStrategy { ... }
9 class LogNormalization : INormalizationStrategy { ... }
```

6.5. Patrón Factory Method

Centraliza la creación de objetos complejos:

```
1 class DistanceFactory
2 {
3     public static IDistanceStrategy CreateDistance(string type)
4     {
5         switch (type.ToLower())
6         {
7             case "euclidean": return new EuclideanDistance();
8             case "manhattan": return new ManhattanDistance();
9             case "cosine": return new CosineDistance();
10            case "hamming": return new HammingDistance();
11            default: throw new ArgumentException("Tipo invalido");
12        }
13    }
14 }
```

7. Estructuras de Datos Implementadas

7.1. CustomList (Lista Enlazada)

Propósito: Almacenar colecciones dinámicas de películas y clusters.

Estructura:

- Cada nodo contiene un dato y referencia al siguiente
- Mantiene referencias a cabeza y cola
- Contador de tamaño

Operaciones principales:

- `Add(T item)`: $O(1)$ - Agregar al final
- `GetAt(int index)`: $O(n)$ - Acceso por índice
- `RemoveAt(int index)`: $O(n)$ - Eliminación

Ventajas:

- Inserción/eliminación eficiente al principio/final
- Crecimiento dinámico sin redimensionamiento
- No desperdicia memoria

7.2. CustomVector (Arreglo Dinámico)

Propósito: Representar vectores de características de películas.

Estrategia de crecimiento: Duplica capacidad cuando está lleno.

Operaciones principales:

- `Add(T item)`: $O(1)$ amortizado
- `GetAt(int index)`: $O(1)$ - Acceso directo
- `SetAt(int index, T value)`: $O(1)$

Ventajas:

- Acceso aleatorio en tiempo constante
- Cache-friendly (datos contiguos)
- Ideal para operaciones matemáticas vectoriales

7.3. CustomMatrix (Matriz Bidimensional)

Propósito: Almacenar matriz de distancias entre películas.

Implementación: Array bidimensional $n \times n$ donde n es el número de películas.

Operaciones:

- `GetAt(int row, int col): O(1)`
- `SetAt(int row, int col, T value): O(1)`

Optimización: Solo se calcula y almacena la mitad superior de la matriz (simétrica).

7.4. CustomHashMap (Tabla Hash)

Propósito: Indexar categorías únicas (géneros, actores, directores).

Características:

- Resolución de colisiones por encadenamiento
- Factor de carga: 0.75
- Redimensionamiento automático

Operaciones:

- `Put(TKey key, TValue value): O(1)` promedio
- `Get(TKey key): O(1)` promedio
- `ContainsKey(TKey key): O(1)` promedio

Uso: Mapear cadenas de texto a índices numéricos para vectorización one-hot.

8. Análisis de Complejidad Algorítmica

8.1. Parseo de CSV

Complejidad: $O(n \cdot m)$

- n : Número de películas
- m : Número promedio de campos por película
- Operaciones dominantes: lectura y parsing de strings

Algorithm 1 Parseo de archivo CSV

Entrada: Archivo CSV con n películas**Salida:** Lista de películas parseadas

```
1: Abrir archivo CSV
2: for cada línea del archivo do
3:   Separar campos por delimitador
4:   Crear objeto Movie
5:   Asignar valores a atributos
6:   Agregar película a lista
7: end for
8: return lista de películas
```

Algorithm 2 Vectorización de dataset

Entrada: Lista de n películas**Salida:** Películas con vectores de características

```
1: Construir índices de categorías  $\triangleright O(n \cdot c)$ 
2: for cada película  $i$  en  $[1, n]$  do
3:   Agregar características numéricas  $\triangleright O(1)$ 
4:   for cada categoría  $j$  do
5:     Crear vector one-hot  $\triangleright O(|categorías_j|)$ 
6:     Agregar a vector de película
7:   end for
8: end for
```

8.2. Vectorización de Películas

Complejidad: $O(n \cdot (k + \sum_{j=1}^c |C_j|))$

- n : Número de películas
- k : Características numéricas (constante = 7)
- c : Número de categorías (constante = 6)
- $|C_j|$: Tamaño del diccionario de categoría j

8.3. Normalización del Dataset

Algorithm 3 Normalización de variables

Entrada: Lista de películas, índice de característica

Salida: Valores normalizados

```

1: Fase 1: Cálculo de estadísticas
2:  $min \leftarrow \infty, max \leftarrow -\infty$ 
3: for cada película do
4:    $valor \leftarrow \text{película.vector}[\text{índice}]$ 
5:    $min \leftarrow \min(min, valor)$ 
6:    $max \leftarrow \max(max, valor)$ 
7: end for
8:
9: Fase 2: Normalización
10: for cada película do
11:    $valor \leftarrow \text{película.vector}[\text{índice}]$ 
12:    $normalizado \leftarrow \frac{valor - min}{max - min}$ 
13:    $\text{película.vectorNormalizado}[\text{índice}] \leftarrow normalizado$ 
14: end for
```

Complejidad: $O(n \cdot k)$

- n : Número de películas
- k : Número de características numéricas
- Dos pasadas sobre el dataset

8.4. Construcción de Matriz de Distancias

Complejidad: $O(n^2 \cdot d)$

- n : Número de películas
- d : Dimensionalidad de los vectores
- Calcula $\frac{n(n-1)}{2}$ distancias

Algorithm 4 Construcción de matriz de distancias**Entrada:** Lista de n películas, estrategia de distancia**Salida:** Matriz de distancias $n \times n$

```

1: Crear matriz  $M$  de tamaño  $n \times n$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:   for  $j \leftarrow i + 1$  to  $n - 1$  do
4:      $v_1 \leftarrow \text{película}[i].\text{vectorPonderado}$ 
5:      $v_2 \leftarrow \text{película}[j].\text{vectorPonderado}$ 
6:      $M[i][j] \leftarrow \text{calcularDistancia}(v_1, v_2)$ 
7:      $M[j][i] \leftarrow M[i][j]$  ▷ Simetría
8:   end for
9:    $M[i][i] \leftarrow 0$ 
10: end for
11: return  $M$ 

```

8.5. Algoritmo de Clustering Jerárquico

Algorithm 5 Hierarchical Agglomerative Clustering**Entrada:** Matriz de distancias M , lista de clusters**Salida:** Cluster raíz del dendrograma

```

1: while número de clusters  $> 1$  do
2:    $(i, j, dist) \leftarrow \text{encontrarClustersMasCercanos}(M)$ 
3:    $nuevoCluster \leftarrow \text{fusionar}(\text{cluster}[i], \text{cluster}[j], dist)$ 
4:   Actualizar matriz eliminando filas/columnas  $i, j$ 
5:   Calcular distancias a nuevo cluster (Average Linkage)
6:   Agregar nueva fila/columna para  $nuevoCluster$ 
7:   Eliminar clusters  $i$  y  $j$  de lista
8:   Agregar  $nuevoCluster$  a lista
9: end while
10: return único cluster restante

```

Complejidad: $O(n^3)$

- Iteraciones: $n - 1$ fusiones
- Por iteración: búsqueda de mínimo $O(n^2)$ + actualización $O(n)$
- Complejidad total: $O(n \cdot n^2) = O(n^3)$

Optimización posible: Uso de heap para encontrar mínimo reduciría a $O(n^2 \log n)$.

8.6. Average Linkage - Cálculo de Distancia

La distancia entre un nuevo cluster fusionado C_{AB} y otro cluster C_k se calcula mediante:**Complejidad:** $O(1)$ - Operación de tiempo constante.

Algorithm 6 Average Linkage - Distancia entre clusters

Entrada: Cluster A , Cluster B , Cluster k , Matriz M **Salida:** Distancia promedio ponderada

```

1:  $size_A \leftarrow$  número de elementos en  $A$ 
2:  $size_B \leftarrow$  número de elementos en  $B$ 
3:  $d_A \leftarrow M[A][k]$  ▷ Distancia  $A$  a  $k$ 
4:  $d_B \leftarrow M[B][k]$  ▷ Distancia  $B$  a  $k$ 
5:  $d_{AB,k} \leftarrow \frac{size_A \cdot d_A + size_B \cdot d_B}{size_A + size_B}$ 
6: return  $d_{AB,k}$ 

```

8.7. Exportación a JSON

Algorithm 7 Conversión de árbol a JSON

Entrada: Nodo raíz del dendrograma**Salida:** Objeto JSON serializado

```

1: function CONVERTIRAJSON( $cluster$ )
2:   if  $cluster = null$  then
3:     return  $null$ 
4:   end if
5:   Crear objeto JSON vacío
6:    $json.distancia \leftarrow cluster.distancia$ 
7:   if  $cluster$  es hoja then
8:      $json.nombre \leftarrow cluster.película.título$ 
9:   else
10:     $json.nombre \leftarrow$ 
11:     $json.hijos.agregar(ConvertirAJson(cluster.izquierdo))$ 
12:     $json.hijos.agregar(ConvertirAJson(cluster.derecho))$ 
13:   end if
14:   return  $json$ 
15: end function

```

Complejidad: $O(n)$

- Recorrido en profundidad del árbol
- Visita cada nodo exactamente una vez
- n nodos (hojas) $\Rightarrow 2n - 1$ nodos totales

9. Construcción del Dendrograma

9.1. Matriz de Distancias

La matriz de distancias es una estructura fundamental que almacena las distancias calculadas entre todos los pares de películas del dataset.

Propiedades:

- Matriz cuadrada de tamaño $n \times n$ donde n es el número de películas
- **Simétrica:** $M[i][j] = M[j][i]$
- **Diagonal cero:** $M[i][i] = 0$ (distancia de un elemento a sí mismo)

Optimización de acceso: Para evitar errores al acceder a la matriz simétrica, siempre se debe garantizar que se acceda usando (min, max) como índices:

```

1 double GetSymmetricDistance(Matrix M, int i, int j)
2 {
3     if (i == j) return 0;
4     if (i < j) return M[i][j];
5     else return M[j][i];
6 }

```

9.2. Proceso de Fusión de Clusters

El algoritmo mantiene una lista activa de clusters y una matriz de distancias que se actualiza en cada iteración:

1. **Inicialización:** Cada película es un cluster individual
2. **Búsqueda:** Encontrar el par de clusters (i, j) con menor distancia
3. **Fusión:** Crear nuevo cluster que contiene ambos clusters
4. **Actualización:**
 - Crear nueva matriz reducida
 - Calcular distancias del nuevo cluster a todos los demás usando Average Linkage
 - Copiar distancias entre clusters no fusionados
5. **Repetir:** Hasta que quede un solo cluster

9.3. Average Linkage

El método Average Linkage calcula la distancia entre clusters como el promedio ponderado de las distancias:

Fórmula:

$$d(C_{AB}, C_k) = \frac{|C_A| \cdot d(C_A, C_k) + |C_B| \cdot d(C_B, C_k)}{|C_A| + |C_B|} \quad (9)$$

Ventajas:

- Menos sensible a outliers que Single Linkage
- Produce clusters más compactos que Complete Linkage
- Balance entre diferentes criterios de linkage

Interpretación: Los clusters más grandes tienen mayor peso en el cálculo de la nueva distancia, lo cual es intuitivo ya que representan más elementos.

9.4. Reconstrucción de la Matriz

En cada iteración, la matriz se reduce de tamaño $n \times n$ a $(n - 1) \times (n - 1)$:

Proceso:

1. La primera fila/columna corresponde al nuevo cluster fusionado
2. Las demás filas/columnas corresponden a clusters no fusionados
3. Se calculan distancias del nuevo cluster a todos los demás
4. Se copian distancias entre clusters existentes

Complejidad espacial: En cada iteración se crea una nueva matriz, lo cual es $O(n^2)$ por iteración, pero como hay n iteraciones, el espacio total usado es $O(n^3)$ acumulado.

10. Sistema de Configuración

10.1. ConfigurationManager (Singleton)

El ConfigurationManager centraliza toda la configuración del sistema de clustering mediante el patrón Singleton.

Responsabilidades:

- Gestión de pesos para cada característica
- Selección de estrategias de normalización
- Selección de métrica de distancia
- Habilitación/deshabilitación de variables

Vector de pesos: Mantiene un vector dinámico donde cada posición corresponde a una característica:

- Posiciones 0-6: Variables numéricas (Budget, Popularity, etc.)
- Posiciones 7+: Variables categóricas expandidas (one-hot encoding)

10.2. Aplicación de Pesos

Después de la normalización, cada valor en el vector de características se multiplica por su peso correspondiente:

$$v'_i = v_i \cdot w_i \quad (10)$$

donde v_i es el valor normalizado y w_i es el peso configurado.

Efecto: Permite controlar la importancia relativa de cada característica en el cálculo de distancias. Un peso de 0 elimina completamente la característica del análisis.

10.3. Configuración de Variables Categóricas

Para variables categóricas (géneros, reparto, etc.), un solo peso se aplica a todas las dimensiones del one-hot encoding correspondiente:

```
1 // Ejemplo: Peso de 15.0 para g_neros
2 // Si hay 20 g_neros_nicos, las 20 dimensiones
3 // correspondientes reciben peso 15.0
```

Esto permite dar mayor o menor importancia a categorías completas de información.

11. Operaciones Vectoriales

11.1. Producto Punto

Calcula la suma de productos elemento a elemento:

Algorithm 8 Producto punto de vectores

Entrada: Vectores v_1 y v_2 de dimensión d

Salida: Valor escalar del producto punto

```
1: resultado  $\leftarrow$  0
2: for  $i \leftarrow 0$  to  $d - 1$  do
3:   resultado  $\leftarrow$  resultado +  $v_1[i] \cdot v_2[i]$ 
4: end for
5: return resultado
```

Complejidad: $O(d)$ donde d es la dimensión del vector.

11.2. Norma Euclidiana

Calcula la magnitud de un vector:

Algorithm 9 Norma euclidiana

Entrada: Vector v de dimensión d

Salida: Magnitud del vector

```
1: suma  $\leftarrow$  0
2: for  $i \leftarrow 0$  to  $d - 1$  do
3:   suma  $\leftarrow$  suma +  $v[i]^2$ 
4: end for
5: return  $\sqrt{\text{suma}}$ 
```

Complejidad: $O(d)$ donde d es la dimensión del vector.

11.3. Resta Vectorial

Calcula la diferencia elemento a elemento:

Complejidad: $O(d)$ donde d es la dimensión del vector.

Algorithm 10 Resta de vectores

Entrada: Vectores v_1 y v_2 de dimensión d **Salida:** Vector diferencia

```
1: Crear vector resultado de dimensión  $d$ 
2: for  $i \leftarrow 0$  to  $d - 1$  do
3:    $resultado[i] \leftarrow v_1[i] - v_2[i]$ 
4: end for
5: return resultado
```

12. Interfaz de Usuario

12.1. Diseño de la Interfaz

La interfaz gráfica está desarrollada en WPF (Windows Presentation Foundation) y sigue los principios de diseño moderno:

Componentes principales:

- **Panel de carga:** Selección de archivo CSV
- **Configuración de métrica:** ComboBox para elegir distancia
- **Variables numéricas:** 7 controles con:
 - CheckBox (habilitar/deshabilitar)
 - TextBox (peso)
 - ComboBox (estrategia de normalización)
- **Variables categóricas:** 6 controles con:
 - CheckBox (habilitar/deshabilitar)
 - TextBox (peso)
- **Botón de ejecución:** Inicia el proceso de clustering
- **Barra de estado:** Muestra progreso y mensajes

12.2. Flujo de Interacción

1. Usuario carga archivo CSV
2. Sistema valida y muestra información del dataset
3. Usuario configura:
 - Métrica de distancia
 - Normalización por variable

- Pesos por variable
 - Variables activas
4. Usuario ejecuta clustering
 5. Sistema procesa y muestra progreso
 6. Sistema exporta resultado a JSON
 7. Usuario recibe confirmación

13. Formato de Exportación JSON

13.1. Estructura del JSON

El dendrograma se exporta en formato JSON con la siguiente estructura:

```
1 {  
2   "n": "T tulo de pel cula o espacio",  
3   "d": 0.0,  
4   "c": [  
5     {  
6       "n": "pel cula1",  
7       "d": 0.0,  
8       "c": []  
9     },  
10    {  
11      "n": "pel cula2",  
12      "d": 0.0,  
13      "c": []  
14    }  
15  ]  
16 }
```

Campos:

- **n:** Nombre (título de película para hojas, vacío para nodos internos)
- **d:** Distancia a la cual se fusionó este cluster
- **c:** Array de hijos (vacío para hojas)

13.2. Características del Formato

- **Jerárquico:** Refleja estructura de árbol
- **Recursoivo:** Cada nodo tiene la misma estructura
- **Portable:** Estándar JSON compatible con cualquier visualizador
- **Legible:** Formato indentado para inspección manual

14. Pruebas y Validación

14.1. Casos de Prueba

14.1.1. Prueba 1: Dataset Pequeño

Objetivo: Verificar correctitud básica del algoritmo.

Configuración:

- 5 películas
- Distancia euclidiana
- Normalización MinMax
- Todos los pesos = 1.0

Resultado esperado: Dendrograma con 4 fusiones, altura creciente.

14.1.2. Prueba 2: Comparación de Métricas

Objetivo: Verificar que diferentes métricas producen resultados diferentes.

Configuración:

- Mismo dataset
- 4 métricas distintas
- Misma configuración de pesos

Resultado esperado: Dendrogramas con diferentes estructuras y alturas.

14.1.3. Prueba 3: Efecto de Ponderación

Objetivo: Verificar que los pesos afectan el clustering.

Configuración:

- Dataset de películas
- Peso alto para género (15.0)
- Peso bajo para otras variables (1.0)

Resultado esperado: Películas del mismo género se agrupan primero.

14.2. Validación de Resultados

Criterios de validación:

1. Todas las películas aparecen en el dendrograma
2. No hay duplicados
3. Distancias son no negativas
4. Distancias aumentan con la altura del árbol
5. JSON es bien formado y parseable

15. Optimizaciones Implementadas

15.1. Vectorización One-Hot Eficiente

En lugar de crear vectores completos para cada categoría por separado, se construye el vector completo en una sola pasada:

Beneficio: Reduce creaciones de objetos y mejora localidad de caché.

15.2. Matriz Simétrica

Solo se calcula la mitad superior de la matriz de distancias, aprovechando la simetría:

Ahorro: $\frac{n(n-1)}{2}$ cálculos en lugar de n^2 .

15.3. Crecimiento Exponencial de Vectores

Los vectores dinámicos duplican su capacidad en lugar de crecer linealmente:

Beneficio: Amortiza el costo de redimensionamiento a $O(1)$ por inserción.

16. Limitaciones y Trabajo Futuro

16.1. Limitaciones Actuales

1. **Complejidad temporal:** $O(n^3)$ limita el tamaño del dataset
2. **Memoria:** Almacenamiento completo de matriz de distancias
3. **Visualización:** No hay visor integrado del dendrograma
4. **Tiempo:** Gran carga academica de otros cursos

17. Conclusiones

Este proyecto ha implementado exitosamente un sistema completo de clustering jerárquico aplicado al análisis de películas, cumpliendo con todos los requisitos establecidos:

1. **Estructuras de datos propias:** Se implementaron desde cero listas enlazadas, vectores dinámicos, matrices y tablas hash, garantizando control total sobre el comportamiento y rendimiento.
2. **Patrones de diseño:** La aplicación de MVC, Singleton, Strategy, Iterator y Factory Method resultó en un código modular, mantenible y extensible.
3. **Flexibilidad:** El sistema permite configurar métricas de distancia, estrategias de normalización y ponderación de variables, adaptándose a diferentes necesidades de análisis.
4. **Escalabilidad del diseño:** Aunque la complejidad algorítmica es $O(n^3)$, la arquitectura permite mejoras futuras sin cambios significativos.
5. **Exportación estándar:** El formato JSON facilita la integración con herramientas de visualización externas.

17.1. Aprendizajes Clave

- La importancia de la normalización para evitar dominancia de variables de gran escala
- El impacto de la elección de métrica de distancia en los resultados
- La relevancia de patrones de diseño en sistemas complejos
- El balance entre complejidad algorítmica y claridad de código
- La necesidad de estructuras de datos eficientes en aplicaciones de ciencia de datos

17.2. Aplicabilidad

El sistema desarrollado es aplicable no solo a películas, sino a cualquier dominio que requiera análisis de similitud:

- Recomendación de productos en e-commerce
- Análisis de perfiles de clientes en marketing
- Clasificación de documentos en minería de texto
- Análisis de secuencias genéticas en bioinformática
- Detección de anomalías en seguridad informática